FINAL RESEARCH REPORT

How Much Accuracy Is Worth the Additional Computing Cost?
Investigating the Trade-off Between AI Performance and Computational Resources

Final project audit and results report
Group 119 | Section C

Evidence cut-off: 7 November 2026

Contents

1. Title page 4

2. Abstract 4

3. Keywords 4

4. Introduction 4

5. Research motivation 4

6. Research problem 4

7. Research questions 5

8. Research objectives 5

9. Background and related work 5

10. Theoretical foundations 5

11. Experimental methodology 5

12. Experimental setup 5

13. Datasets 5

14. Model architectures 6

15. Computational budget definitions 6

16. Evaluation metrics 6

17. Statistical methodology 6

18. Experimental results: evidence overview 6

19. Supervised learning analysis 7

20. Unsupervised learning analysis 7

21. Semi-supervised learning analysis 7

22. Reinforcement learning analysis 7

23. Language-model analysis 8

24. Published literature comparison 8

25. Marginal performance improvement analysis 8

26. Accuracy-versus-compute trade-off 8

27. Diminishing returns analysis 8

28. Cross-paradigm discussion 9

29. Limitations 9

30. Threats to validity 9

31. Reproducibility 9

32. Conclusion 9

33. Future research directions 9

34. References 9

35. Appendix: experimental configurations 10

36. Appendix: completion status and additional figures 10

1. Title page

How Much Accuracy Is Worth the Additional Computing Cost? Investigating the Trade-off Between AI Performance and Computational Resources

Final research audit and results report

Research project, Group 119, Section C

Evidence cut-off: 7 November 2026 Repository: `Vatsal-Garg-iiitd/Research-Experiment`

2. Abstract

This report audits the project's existing supervised, unsupervised, semi-supervised, reinforcement-learning (RL), language-model, and literature records. It recomputes descriptive statistics and within-run marginal changes from saved measurements only. No training, inference, benchmarking, or data collection was conducted for this audit. Evidence includes multi-seed Fashion-MNIST learning curves, RL interaction curves, local evaluation of six public language-model checkpoints on WikiText-2, three full zero-shot task evaluations for Pythia-70M, and a field-level literature ledger. The strict project framework records **0 of 50 requested objectives fully complete**. This coexists with substantial partial evidence: 2,548 raw conventional-result rows are indexed, 64 LLM evaluation rows are indexed, and the analysis contains 3,614 grouped metric summaries, 2,417 within-seed marginal transitions, and 169 held-out metric groups. Fifteen duplicate K-Means records were excluded from the analytical trajectory because each repeats an earlier experiment ID and copies the final checkpoint score into earlier budgets; raw values remain untouched. The results show descriptive, task-specific changes with budget, including reversals and plateaus. They do not establish a general diminishing-returns law or a cross-paradigm compute-efficiency ranking. The defensible answer to the research question is therefore conditional: added budget sometimes improves measured performance, but whether its cost is worthwhile depends on task, metric, model, evaluation protocol, and stopping point; current evidence is insufficient for a universal threshold or causal claim.

3. Keywords

Computational cost; marginal performance; scaling; supervised learning; semi-supervised learning; reinforcement learning; language models; reproducibility; research audit.

4. Introduction

Model quality is commonly reported without a commensurate account of the resources used to obtain it. A useful comparison must specify the resource being spent, the outcome metric, the evaluation set, and the method by which additional budget changes the result. This project asks how performance changes as computational expenditure increases and whether marginal improvements diminish.

The repository is an exploratory multi-paradigm collection rather than one controlled experiment. Its evidence includes distinct tasks, budgets, devices, and metrics. We therefore analyze each family in its native units and treat the synthesis as a set of bounded case studies.

5. Research motivation

Training and evaluation can consume substantial compute, time, memory, and energy. Reporting marginal quality per additional resource can help decide when a run is useful to extend. However, wall time is hardware- and environment-dependent, parameter count is not training compute, environment steps are not optimizer updates, and benchmark accuracy is not interchangeable with reconstruction error or language-model perplexity. These distinctions motivate a protocol-specific audit rather than a single pooled score.

6. Research problem

The central problem is to determine whether observed gains justify the added measured budget. The evidence must support paired within-configuration comparisons and preserve negative gains, uncertainty, incomplete curves, and timing anomalies. The study cannot infer costs that were not recorded or treat public checkpoint evaluation as a measurement of original pretraining.

7. Research questions

1. How do task-specific outcomes change over the saved native budget curves?
2. What marginal improvement is observed between adjacent budgets within a seed and configuration?
3. Do these finite observations support diminishing returns or saturation?
4. What conclusions remain unresolved because runs are partial, protocols differ, or resource measurements are missing?

8. Research objectives

The original scope spans 20 conventional methods, 15 LLM configurations, and 15 literature configurations. The current strict all-or-nothing matrix remains 0/50. This criterion is preserved: an experiment can be executed and informative while its requested objective remains incomplete. The audit objective is to verify the evidence and produce a reproducible record, not to increase the completion score retrospectively.

9. Background and related work

Published scaling studies characterize relationships among model size, training tokens, loss, and downstream performance under particular data and optimization regimes. GPT-3 studied few-shot behavior across scales; Gopher reported a 280B model and broad task suite; Chinchilla argued that model size and training tokens should scale together under a compute budget; PaLM reported 540B and 62B configurations; OPT released models through 175B; LLaMA reported four sizes through 65B; and BLOOM documented a 176B multilingual model. These are literature observations, not local replications. They differ in corpus, tokenizer, architecture, training compute definition, benchmark protocols, and reporting coverage. See the evidence classifications in `results/literature/literature_field_evidence.csv` and the citations in Section 34.

10. Theoretical foundations

For a higher-is-better metric (y), the adjacent gain is (\Delta y_i=y_i-y_{i-1}). For a lower-is-better loss (L), favorable improvement is (L_{i-1}-L_i). With native budget (b), a budget-normalized gain is (\Delta y/\Delta b). When positive, the measured time cost per unit improvement is (\Delta t/\Delta y). These ratios are meaningful only within a matched series and with a valid measured time increment. A non-positive gain does not have a finite positive "cost per gain"; it is reported as a reversal or zero gain.

11. Experimental methodology

This is a post hoc audit of existing records. Raw CSV, JSON, JSONL, checkpoint/configuration, and log artifacts remain source evidence. Analysis scripts rebuild the common index, summaries, marginal tables, inventory, and figures. They do not call training or evaluation entry points. Superseded, failed, incomplete, smoke, and alternate protocol records are kept in the repository and separated by status. Duplicate exports with shared experiment IDs are not pooled.

12. Experimental setup

Neural Fashion-MNIST work includes historical Windows/DirectML records and later Apple M3/PyTorch MPS records. Classical methods and RL use CPU records. Platform timings are not pooled as a controlled hardware comparison. Per-run hardware/backend, split identity, seeds, and configuration paths are recorded where available. The old and newer environments remain separately documented in `MACOS_EXECUTION_LOG.md`, `MACOS_SETUP.md`, and `HARDWARE_BENCHMARK_REPORT.md`.

13. Datasets

Most conventional learning curves use Fashion-MNIST with a fixed local 10,000-example training subset and 2,000-example validation subset; official test data are used for selected final endpoints. Requested 60,000-example runs reserve 2,000 examples for validation and optimize on 58,000. DBSCAN uses PCA-32 features and a transductive validation protocol for some sweeps. RL uses CartPole-v1 and Pendulum-v1. Language-model full evaluation uses WikiText-2 raw-v1 test text; zero-shot task results use the validation splits named in the harness record. Published literature rows use paper-specific datasets.

14. Model architectures

The five supervised scope entries are MLP, LeNet-style CNN, ResNet-18, ViT-Tiny, and Random Forest. A genuine ViT-Tiny-dimension VisionTransformer run uses 28x28 grayscale Fashion-MNIST inputs, 4x4 patches, 12 blocks, 192 hidden width, three attention heads, 768-wide MLP, one-channel projection, and a 10-class head (5,353,738 parameters). It is an input-resolution/channel adaptation of canonical ViT-Tiny dimensions. The separate custom three-layer, width-96 transformer is a project proxy and is not counted as the ViT-Tiny implementation. Unsupervised methods include MiniBatch K-Means, Gaussian Mixture, DBSCAN, Autoencoder, and VAE. Semi-supervised methods include pseudo-labeling, Mean Teacher, FixMatch, MixMatch, and actual scikit-learn LabelPropagation. Some earlier LabelSpreading records are superseded and are not renamed as LabelPropagation. RL includes DQN, Double DQN, A2C, PPO, and SAC; PPO-Pendulum is an additional recorded extension, distinct from the five-method original scope.

15. Computational budget definitions

Budgets remain method-specific: epochs/examples for neural supervised and representation models; tree count for Random Forest; partial-fit updates for MiniBatch K-Means; warm-start EM fit calls for GMM; fitted sample count and density parameters for DBSCAN; maximum iterations for LabelPropagation; label fraction and epochs for SSL; environment interactions and gradient updates for RL; released checkpoint step/training-token metadata for Pythia trajectories; and evaluation tokens/examples for inference. No epoch equivalent is invented for non-epoch methods. Training compute for pretrained public LLMs is not measured locally.

16. Evaluation metrics

Classification accuracy, macro precision/recall/F1, and cross-entropy are reported separately. Clustering uses ARI, NMI, silhouette, and where present log-likelihood. Reconstruction uses MSE/objective loss; representation quality uses a separate linear-probe accuracy. RL uses episodic return and its within-evaluation episode dispersion. Language modeling uses cross-entropy/perplexity and bits per byte where recorded. Literature benchmarks retain their published definitions. No cross-metric composite is calculated.

17. Statistical methodology

For repeated runs, means, sample standard deviations, medians, minima, maxima, valid-seed counts, and two-sided 95% Student-t confidence intervals are produced where (n>1). For (n=1), no interval is asserted. These intervals describe seed variability conditional on the fixed data split and are not population-level dataset uncertainty. Marginal transitions pair adjacent budgets within the same model, seed, split, method, label fraction, and source series. A single seed's checkpoints are not independent replicates. No p-values or cross-model significance tests are claimed.

18. Experimental results: evidence overview

The current canonical local index contains 2,612 rows: 2,548 source rows from conventional raw CSVs and 64 rows from selected LLM CSV exports. This is a lossless heterogeneous index, not a count of independent experiments. It includes repeated checkpoints, superseded and partial records. A prior 619-row canonical export is recoverable from the committed historical version at `afc488a`; an execution-log milestone cited 430 rows, but no matching snapshot was found, so exact membership of that intermediate export is unavailable. The old report's 458 grouped summaries, 279 marginal increments, and 84 held-out groups reflect an earlier analysis state; the current reproducible run reports 3,614, 2,417, and 169 respectively. These are different processing generations and must not be added together.

19. Supervised learning analysis

MLP, LeNet-style CNN, ResNet-18, the compact transformer proxy, and the ViT-Tiny-dimension model have 30-epoch validation curves at the fixed 10K train subset, three seeds each. MLP and LeNet also have five-epoch endpoints at 30K and 58K actual train sizes. At five epochs, MLP test accuracy averages 0.8537 (SD 0.0081; 95% t CI [0.8336, 0.8737]) at 30K and 0.8689 (SD 0.0008; CI [0.8669, 0.8710]) at 58K. LeNet averages 0.8784 (SD 0.0035; CI [0.8699, 0.8870]) and 0.8989 (SD 0.0035; CI [0.8903, 0.9075]), respectively. Each size uses three matched seeds. The two points are consistent with better observed accuracy at the larger size, but cannot establish a general data-scaling law. The genuine ViT-Tiny-dimension model's 30-epoch official test accuracy averages 0.7621 (SD 0.0616; 95% t CI [0.6090, 0.9152], min 0.6916, max 0.8057; n=3). This wide interval and range indicate substantial seed variability in this implementation; the result does not support a stable model ranking. Validation trajectories and time plots are Figures 1-5 and 20. Test endpoints at 30 epochs are distinct from the matched five-epoch data-size comparison.

The ResNet timing observation is preserved at 1,122.1625 seconds for the epoch-2-to-3 interval for seed 11. Adjacent timestamps show an approximately 18m45s gap; the next epoch interval is about 9.7s. The observation is a genuine logged wall-clock interval, but available logs cannot distinguish compute from process suspension, interruption, or accounting delay. It is retained in the raw-time plot and excluded from the adjusted efficiency diagnostic. Neither value is silently deleted or asserted to be pure device compute. Figure 20 plots supervised validation accuracy against cumulative measured training time after the same limited adjustment.

20. Unsupervised learning analysis

The raw unsupervised table has 51 rows. Fifteen later K-Means rows repeat earlier experiment IDs and stamp budgets 1, 5, 10, and 20 with the score of the 50-update model state. They are preserved but excluded from the analytical curve; 15 K-Means observations across three seeds remain. The GMM records contain 21 rows across three seeds and distinct iteration budgets. Figures 6-7 plot ARI against native iterative units. Figure 8 shows DBSCAN's fit-size sweep; ARI is near zero in most listed settings and its size points are not treated as an iterative learning curve. Figures 9-10 keep AE/VAE reconstruction MSE separate from downstream linear-probe accuracy. Non-neural methods have no invented epoch counts.

21. Semi-supervised learning analysis

Pseudo-labeling and its labeled-only baseline cover five label fractions with three seeds and short five-epoch curves. Mean Teacher, FixMatch, and MixMatch each have 450 validation rows: five label fractions x 30 epochs x three seeds. Their 90 test rows contain one terminal evaluation per model/fraction/seed. These are grayscale Fashion-MNIST adaptations, not canonical benchmark replications. Figure 11 compares endpoint test accuracy by label fraction; Figure 12 compares validation curves at 10% labels; Figure 21 compares pseudo-labeling with the matched labeled-only MLP baseline at five epochs. Results are not generalized beyond this dataset and recipe. Label Propagation has 30 current completed rows (three seeds x iteration ceilings) and 15 explicitly superseded legacy LabelSpreading/implementation rows; it is reported as LabelPropagation only for the current implementation. Figure 22 plots linear-probe representation quality against measured training time, separately from reconstruction loss.

22. Reinforcement learning analysis

The raw RL file contains 132 rows across six recorded methods. DQN, A2C, and PPO have three seeds at 5K, 10K, 25K, 50K, 100K, and 250K interactions. SAC and Double DQN have five seeds at these checkpoints. The Double DQN rows have `completed` experiment status and `local_measurement` source type after derived-schema alignment. The raw CSV omitted the success_rate cell, so that field remains unavailable; all other shifted fields are realigned in derived outputs without modifying the raw rows. PPO-Pendulum is the additional sixth method, an extension rather than a change to the original five-algorithm scope. CartPole and Pendulum results are plotted separately in Figures 13-14 because their reward definitions and ranges differ. Checkpoint evaluation uses 20 deterministic episodes per record; training and evaluation environments are separate. Requested step budgets, actual steps, overshoot, and a legacy resumed run are detailed in the execution log. The finite and noisy curves do not identify a universally best algorithm or a general saturation point.

23. Language-model analysis

Six public checkpoints were evaluated on the complete WikiText-2 raw-v1 test text on Apple M3 MPS, FP32, context 1,024, stride 512, batch size one. Pythia models scored 285,830 tokens; GPT-2/OPT scored 283,286 because tokenizers differ. Perplexities were Pythia-70M 44.9334, 160M 25.6365, 410M 15.4583, GPT-2 Small 24.9794, OPT-125M 25.8534, and OPT-350M 20.7992. These are single local endpoint evaluations; model-family comparison is limited by tokenizer, training data, and checkpoint differences. Figure 15 presents this matched local evaluation separately from pretraining compute.

Checkpoint-prefix scaling is a different protocol: Pythia-70M and 160M each have eight selected released checkpoints evaluated on a 2,048-token prefix of the first 30,000 raw characters using CPU FP32; Pythia-410M has selected 0, 10K, and 143K points. Selected training-token counts derive from Pythia's per-step metadata and are not measured local compute. Figures 16-17 show the two trajectories, including step 0. They are checkpoint evaluations, not original pretraining runs. Full zero-shot harness evaluation used Pythia-70M step143000, lm-evaluation-harness 0.4.13, FP32 MPS, batch 1, 0-shot/no limit on validation splits: HellaSwag 10,042 examples at 0.2667 accuracy; PIQA 1,838 at 0.5985; ARC-Easy 2,376 at 0.3754. Figure 18 shows task results separately. Earlier 100-example/task Pythia-70M, GPT-2 Small, and OPT-125M runs are smoke tests; Figure 25 presents these restricted scores in its own panel. Figures 23-24 show full-test inference time and loss against model size; they are not training-compute plots. A multi-model full-split zero-shot score-versus-size figure cannot be produced: only Pythia-70M has the full validation-split run. The six full evaluations, prefix curves, benchmark run, old 50K-token tests, smoke tests, failed/resource-limited records, and superseded aliases remain separate in the LLM ledgers.

24. Published literature comparison

The literature ledger contains 15 published configurations and 300 requested model-field rows (20 fields per configuration). Evidence classes are 172 reported, 8 derived, 16 estimated, and 104 uncertain. A field marked uncertain may be absent, not yet extracted, or not established by available evidence. Training tokens, parameter totals, hardware, duration, compute, datasets, and benchmarks are not forced into a common unit. FLOPs and GPU-hours are never conflated without a stated conversion. Literature-only configurations are not counted as locally completed experiments.

25. Marginal performance improvement analysis

The regenerated table contains 2,417 adjacent within-seed transitions. Each row records native budget delta, favorable-direction score change, relative change, additional logged seconds where available, gain per native budget unit, gain per second, and seconds per positive unit gain. Positive, zero, and negative changes are retained. Time-normalized values are not reported where the time delta is nonpositive or missing. Table-level summaries and row-level results are in `results/processed/strengthened_marginal_gains.csv`; the complete discussion is in `MARGINAL_GAIN_ANALYSIS.md`.

26. Accuracy-versus-compute trade-off

Within comparable local runs, marginal ratios can characterize a specific checkpoint extension, but they do not price compute universally. Epoch seconds vary with device/backend and host load; RL seconds depend on environment and implementation; and no original LLM pretraining times are available for local comparison. ResNet's anomalous interval demonstrates why measured elapsed wall time must not be equated automatically with accelerator work. The most defensible operational output is a task- and protocol-specific curve plus its measured incremental cost, not a single cross-project exchange rate.

27. Diminishing returns analysis

The pipeline screens for two consecutive nonnegative relative improvements below 0.5%, 1%, or 2% within a series. A screen can flag small observed increments but does not establish saturation: sequences are short, threshold choice is conventional, some curves are noisy or negative, and (n=3) often gives wide uncertainty. An isolated interval or negative gain is not evidence of a general diminishing-returns law. Current evidence cannot establish the central diminishing-returns hypothesis across paradigms. It supports only local descriptive statements about finite observed trajectories.

28. Cross-paradigm discussion

Figure 19 displays percent change from each series' first recorded point in separate panels for supervised accuracy, semi-supervised accuracy, CartPole return, and reconstruction MSE. It uses an ordered native-budget index rather than a shared cost axis. This normalization shows within-series direction only; it does not make percentage changes comparable across metrics, baselines, or budgets. A robust cross-paradigm cost-efficiency ranking is not supported.

29. Limitations

The work is incomplete against its 50-objective framework. Several intended model configurations are unavailable or partial. Many conventional curves have only three seeds and fixed splits; SSL recipes are task adaptations; RL rewards are noisy and seed counts differ; model-selection/test protocols are not perfectly uniform across historical outputs. Energy and peak-memory coverage is incomplete. Local full-test LLM evidence covers six checkpoints, whereas many planned model sizes were not evaluated. The literature ledger has 104 uncertain fields. No new experiment is authorized or needed to state these limits.

30. Threats to validity

Internal validity is limited by heterogeneous code paths, platform changes, historical superseded runs, wall-clock anomalies, and incomplete logs. Construct validity is limited by treating time as a resource proxy when compute/energy are not measured and by using task-specific metrics. External validity is limited by Fashion-MNIST subsets, small seed counts, one RL environment per action regime, and one local WikiText-2 protocol. Reproducibility risks include evolving dependencies, checkpoint aliases, incomplete original hardware metadata, and data transformations whose configurations must be read with each source record.

31. Reproducibility

From the project root, rebuild derived records without invoking model execution:

```
.venv-mac/bin/python scripts/build_canonical_results.py
.venv-mac/bin/python scripts/analyze_strengthened_runs.py
.venv-mac/bin/python scripts/analyze_llm_full_eval.py
.venv-mac/bin/python scripts/build_literature_evidence.py
.venv-mac/bin/python scripts/plot_final_figures.py
.venv-mac/bin/python scripts/audit_repository_inventory.py
.venv-mac/bin/python scripts/validate_project_outputs.py
```

These commands read existing records and regenerate indices, statistics, figures, identifier reconciliation, and inventory. They do not train or infer. Raw data and source snapshots are not rewritten. See `REPRODUCIBILITY.md` for environment and provenance detail.

32. Conclusion

Additional native budget sometimes coincides with better measured performance, especially over early supervised epochs and larger MLP/LeNet training subsets; later increments can be small, variable, or negative. The current evidence does not identify a universal amount of accuracy worth a unit of compute and does not prove general diminishing returns. The defensible conclusion is that the trade-off is conditional on task, model, protocol, metric, hardware, and the chosen endpoint. Broader causal or universal claims remain unresolved.

33. Future research directions

Future work, if separately authorized, could pre-register comparable budgets, replicate matched conditions with more seeds, record energy and device utilization, standardize validation/test selection, and evaluate representative LLM checkpoints under a fully matched protocol. These are recommendations only; this audit performs none of them and does not complete missing runs.

34. References

1. Brown et al. (2020). "Language Models are Few-Shot Learners." arXiv:2005.14165, especially model sizes and training tokens in Table 2.1.

2. Rae et al. (2021). "Scaling Language Models: Methods, Analysis & Insights from Training Gopher." arXiv:2112.11446.
3. Hoffmann et al. (2022). "Training Compute-Optimal Large Language Models." arXiv:2203.15556.
4. Chowdhery et al. (2022). "PaLM: Scaling Language Modeling with Pathways." arXiv:2204.02311.
5. Zhang et al. (2022). "OPT: Open Pre-trained Transformer Language Models." arXiv:2205.01068.
6. Touvron et al. (2023). "LLaMA: Open and Efficient Foundation Language Models." arXiv:2302.13971.
7. Scao et al. (2023). "BLOOM: A 176B-Parameter Open-Access Multilingual Language Model." JMLR 24(1).
8. WikiText-2 raw-v1, Salesforce Research/Hugging Face dataset repository, revision recorded in `results/llm/macos_full_wikitext2_evaluations.csv`.

Field-level source URLs, locators, and evidence classes are in `results/literature/literature_field_evidence.csv` and `results/literature/literature_sources.md`. Primary-paper field verification is summarized in `LITERATURE_VALIDATION_REPORT.md`.

35. Appendix: experimental configurations

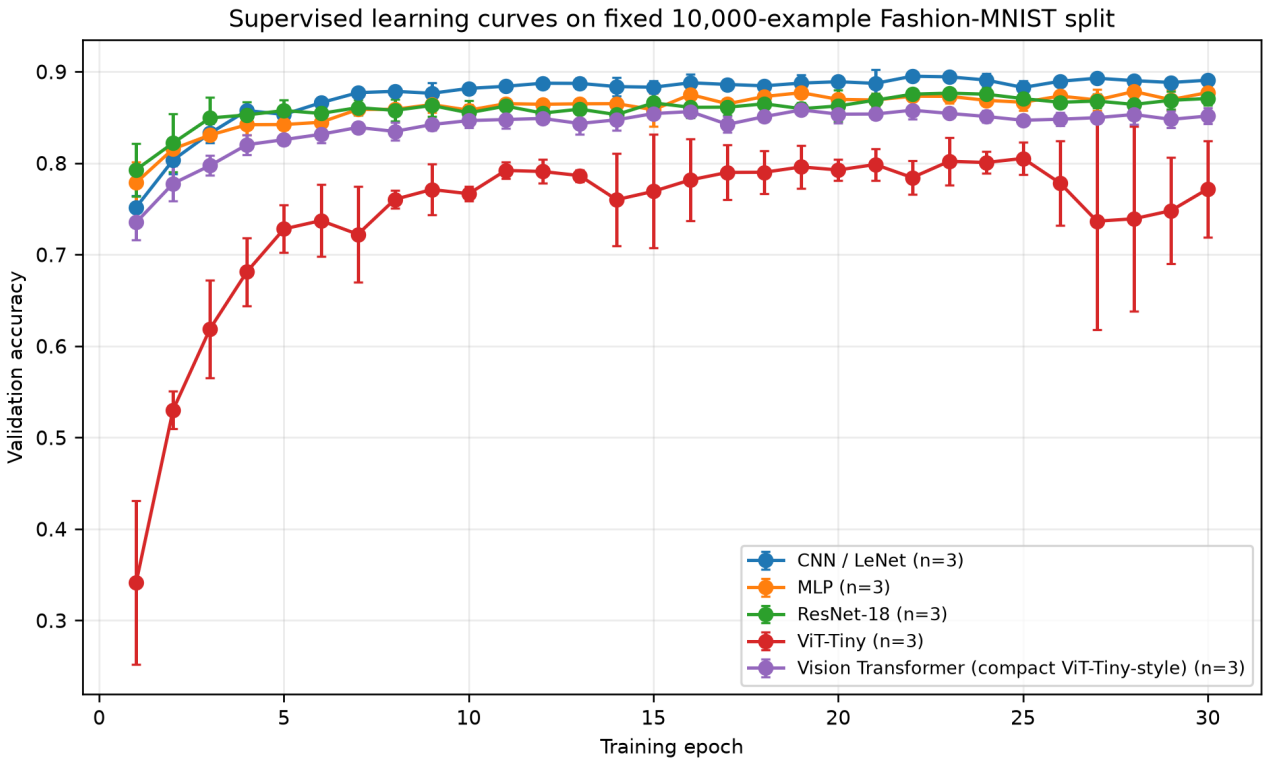
The experiment run registry and `results/configurations/*.json` record run-level model, seed, data split, budget, backend, and source references where available. The main fixed Fashion-MNIST split is identified by split ID `24574ee00a20ab31`. Primary budgets are: 1-30 epochs for main supervised/AE/VAE/SSL curves; 10-1,000 trees for Random Forest; native update/iteration schedules for clustering and LabelPropagation; 5K-250K requested environment steps for RL; and protocol-specific checkpoint and evaluation lengths for LLMs. Label fractions for the multi-method SSL study are 1%, 5%, 10%, 20%, and 40%. Detailed hyperparameters remain in per-run JSON rather than being reconstructed from memory. Missing configuration evidence is marked unavailable in the inventory and audit register. `results/processed/canonical_results.csv` preserves a common source index; `results/processed/verified_canonical_results.csv` adds explicit protocol-level eligibility and exclusion reasons without combining unlike metrics. `IDENTIFIER_AUDIT.md` and `IDENTIFIER_RECONCILIATION.csv` provide source-scoped uniqueness, foreign-key checks, and all derived identifier corrections.

36. Appendix: completion status and additional figures

The strict completion denominator remains 50: 20 conventional, 15 LLM, 15 literature objectives. Fully complete objectives: 0; the matrix still records partial work and statuses per model. Evidence amount, partial objectives, and complete objectives are different quantities. The 0/50 result means none met all registered criteria, not that no work was performed. Figure paths, sources, scripts, interpretation, and limitations for 25 figures are listed in `FIGURE_CATALOGUE.md`; PNG and vector SVG files are under `results/figures/final/`. Inventory and row-level schema/missingness summaries are in `DATA_INVENTORY.md` and `results/processed/tabular_data_inventory.json`. Identifier and foreign-key audit outputs are in `IDENTIFIER_AUDIT.md` and `results/processed/`.

Figure appendix

Figure 1. Supervised validation accuracy versus epoch

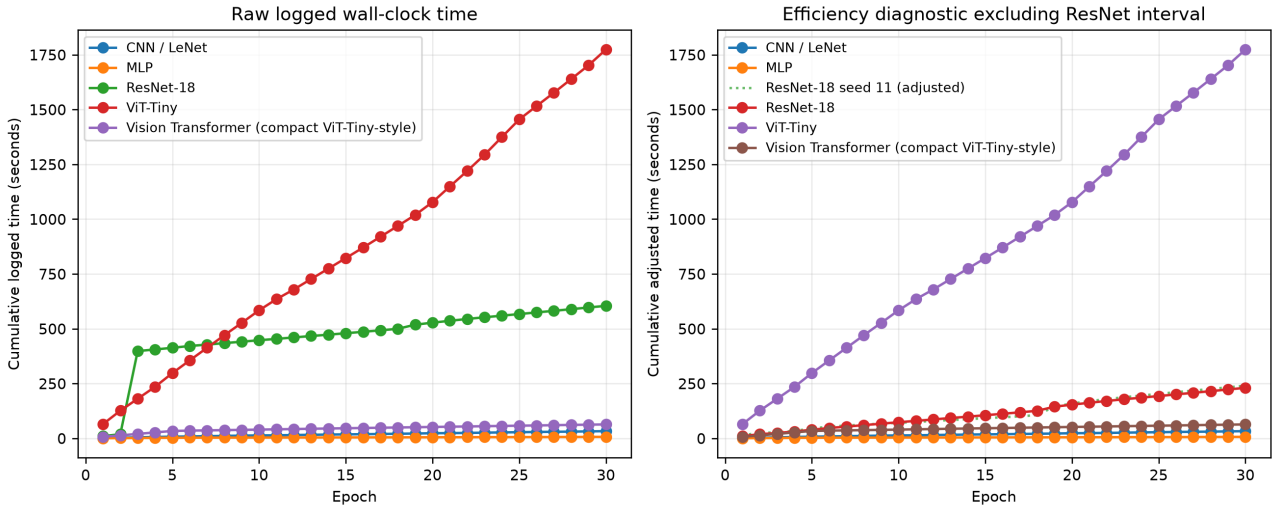


Data: results/raw/replicated_scaling_results.csv; results/raw/vit_tiny_scaling.csv

Interpretation: Accuracy generally increases early, with model-specific later changes; the actual ViT-Tiny-dimension run is shown separately from the compact proxy.

Limitations: Three seeds; standard deviation bars; the 12-block ViT-Tiny has notably variable terminal test results.

Figure 2. Training time versus epoch, raw and adjusted

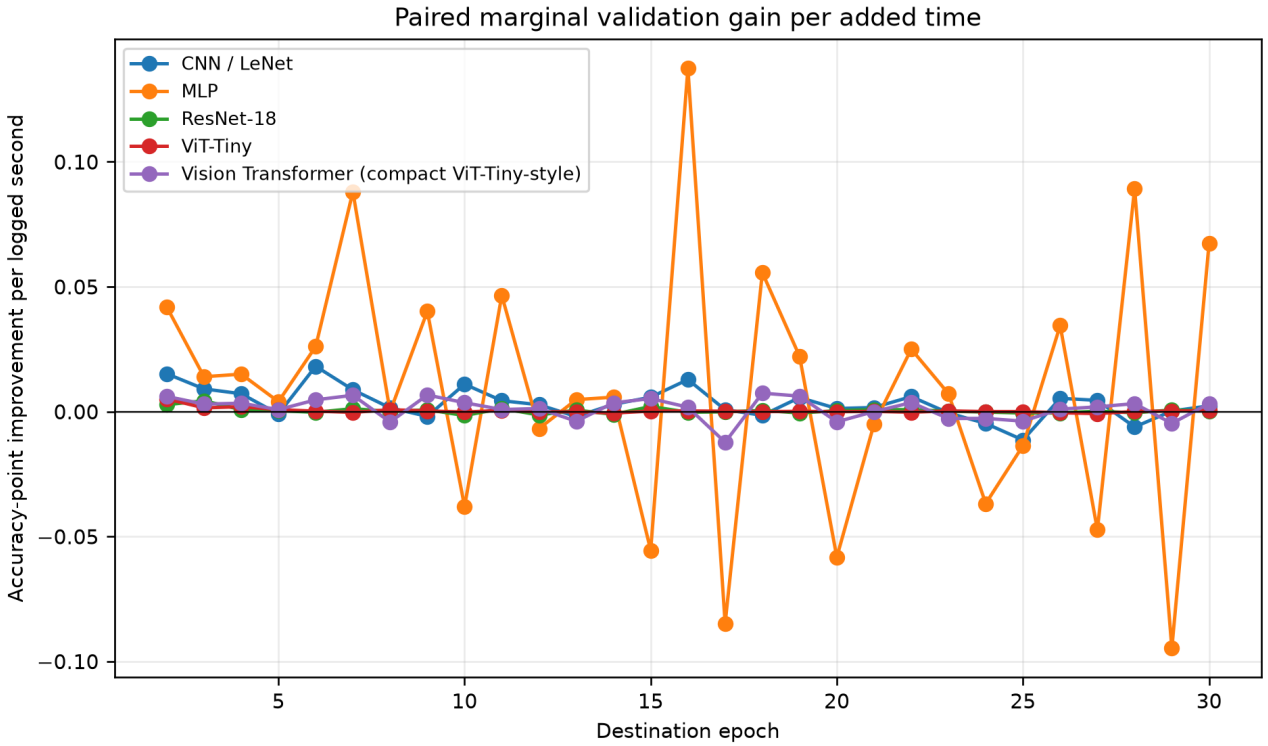


Data: results/raw/replicated_scaling_results.csv

Interpretation: The ResNet epoch-2-to-3 interval dominates raw time; adjusted diagnostic removes 1,122.16 s from seed 11 after epoch 2.

Limitations: Adjustment is analytical only, not a claim about true compute duration; no process logs identify the pause.

Figure 3. Marginal accuracy improvement per additional compute

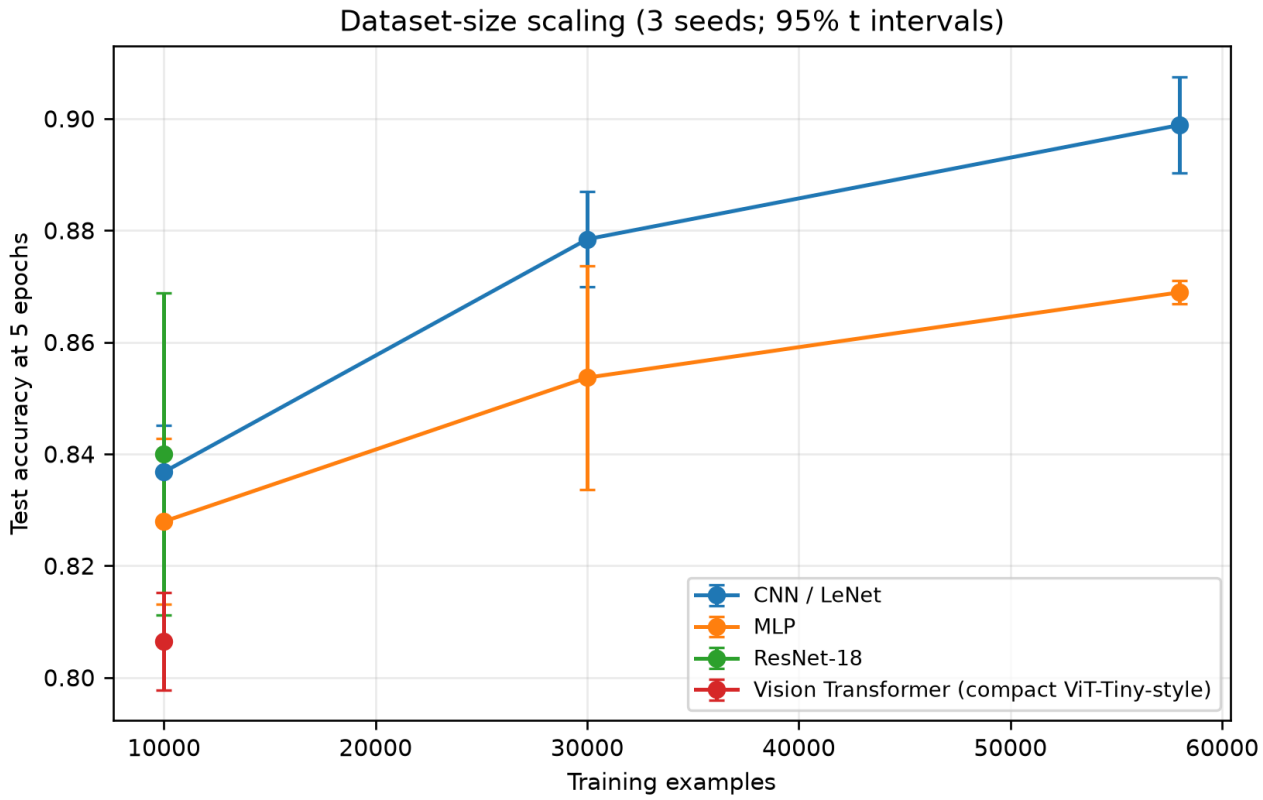


Data: results/processed/strengthened_marginal_gains.csv

Interpretation: Marginal returns fluctuate and can be negative.

Limitations: Per-second metric is sensitive to wall-clock noise; ResNet anomaly interval is excluded in adjusted calculation only if available in summary.

Figure 4. Dataset-size scaling for MLP and LeNet

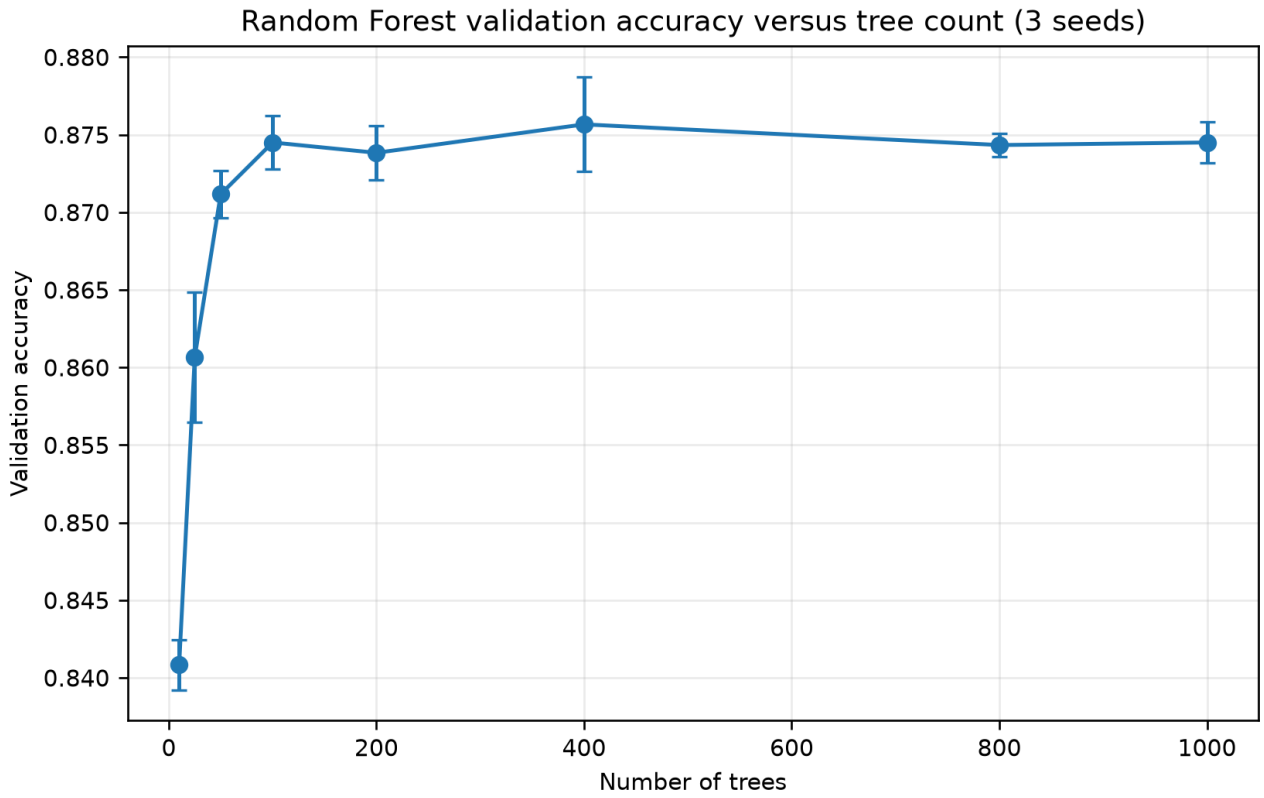


Data: results/processed/data_size_scaling_summary.csv

Interpretation: LeNet increases across the two reported sizes; MLP is not monotone across its available range.

Limitations: Only 30K and 58K requested-size points; intervals with three seeds are imprecise.

Figure 5. Random Forest performance versus number of trees

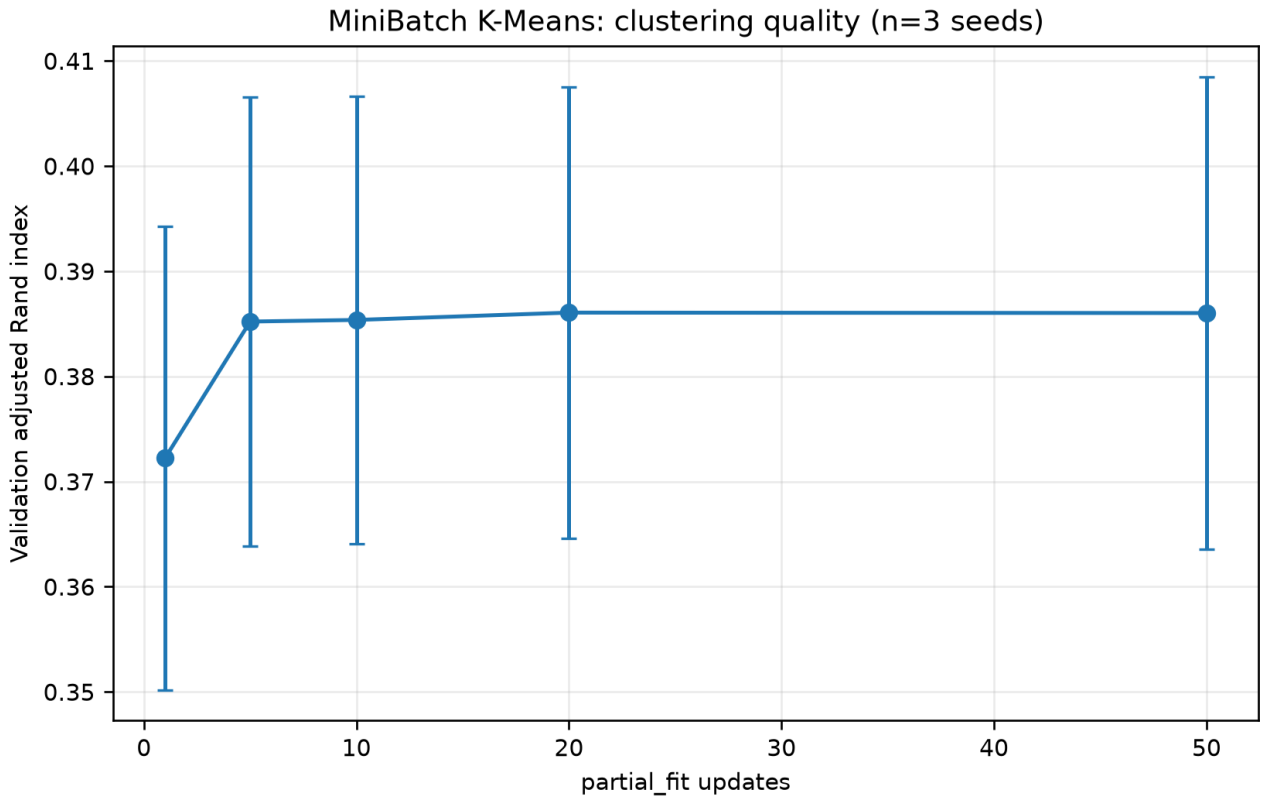


Data: results/raw/random_forest_scaling.csv

Interpretation: The curve provides a native model-size budget comparison.

Limitations: Warm-start forest runs; no interpretation as neural epochs.

Figure 6. MiniBatch K-Means performance versus computational budget



Data: results/raw/unsupervised_scaling.csv

Interpretation: ARI changes over the algorithm-specific iterative budget.

Limitations: K-Means reused-checkpoint duplicate rows are excluded; no epoch equivalence is implied.

Figure 7. Gaussian Mixture Model performance versus computational budget

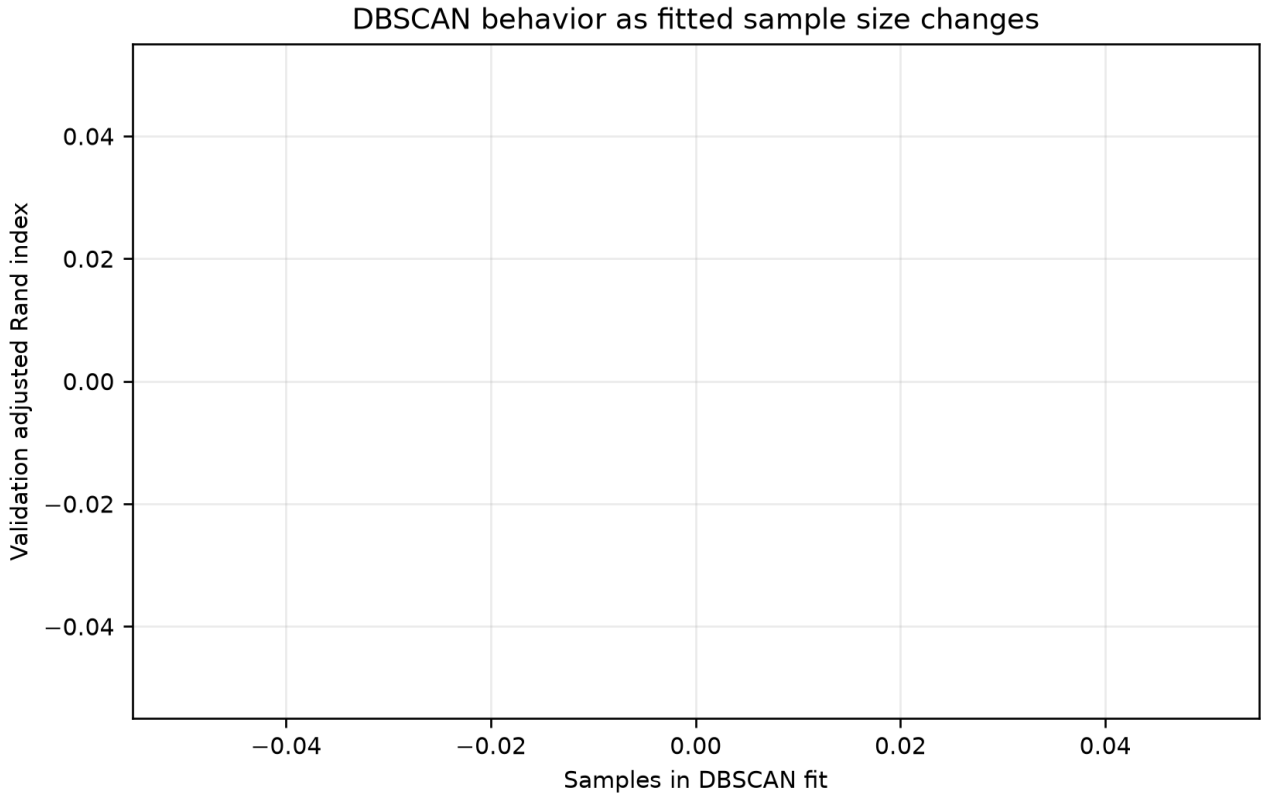


Data: results/raw/unsupervised_scaling.csv

Interpretation: ARI changes over the algorithm-specific iterative budget.

Limitations: K-Means reused-checkpoint duplicate rows are excluded; no epoch equivalence is implied.

Figure 8. DBSCAN performance versus dataset size

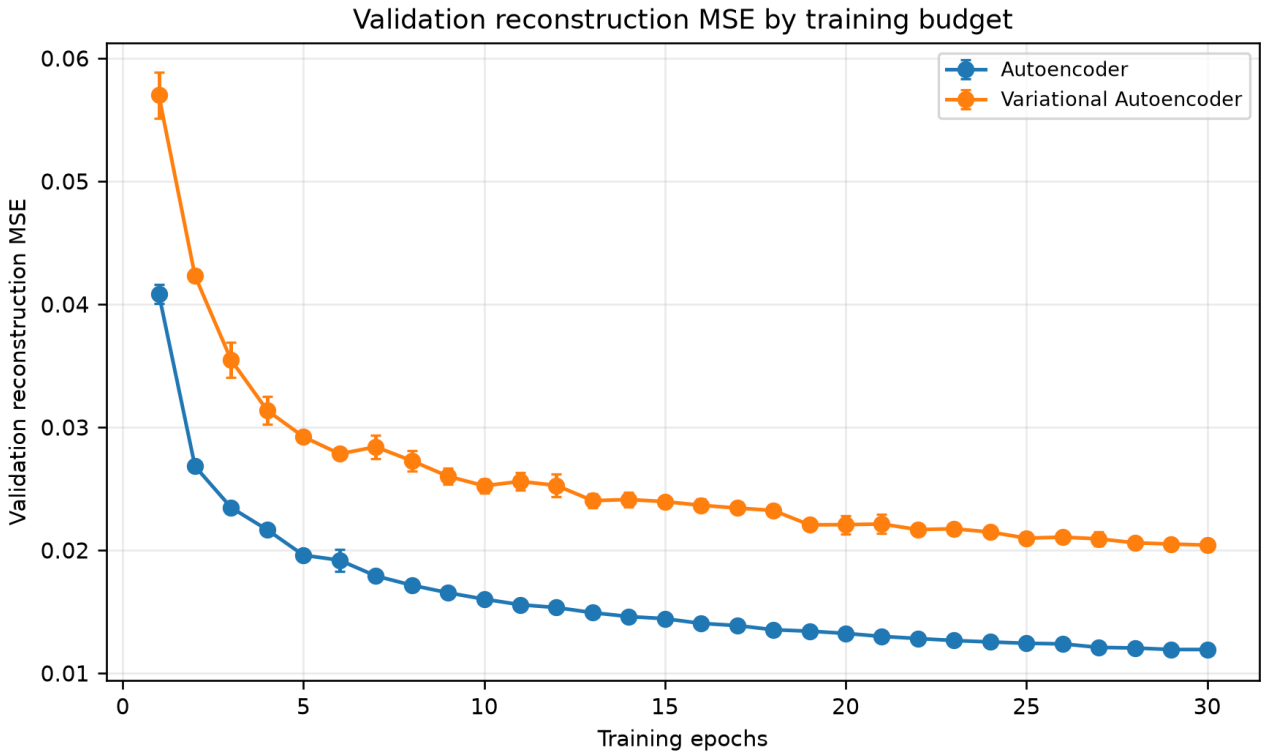


Data: results/raw/dbscan_size_scaling.csv

Interpretation: The size sweep tests data-fit cost with fixed density settings.

Limitations: Nine observations are parameter and budget conditions; no replicated seed error bars; mostly near-zero ARI per existing records.

Figure 9. Validation reconstruction MSE versus training budget



Data: results/raw/autoencoder_scaling.csv

Interpretation: Validation reconstruction MSE is reported separately for each representation model.

Limitations: Three seeds; reconstruction and downstream metrics answer different questions.

Figure 10. Downstream linear-probe accuracy versus training budget

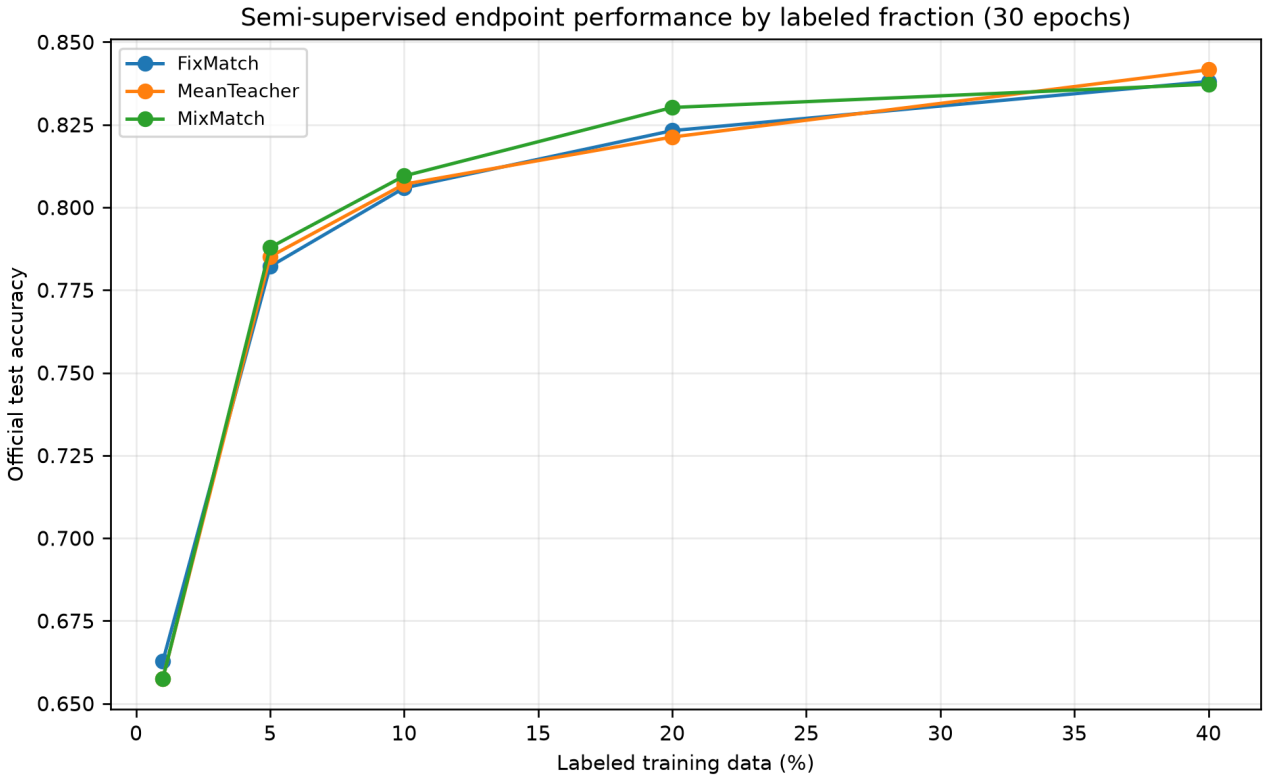


Data: results/raw/autoencoder_scaling.csv

Interpretation: Downstream linear-probe accuracy is reported separately for each representation model.

Limitations: Three seeds; reconstruction and downstream metrics answer different questions.

Figure 11. Semi-supervised performance versus labeled fraction

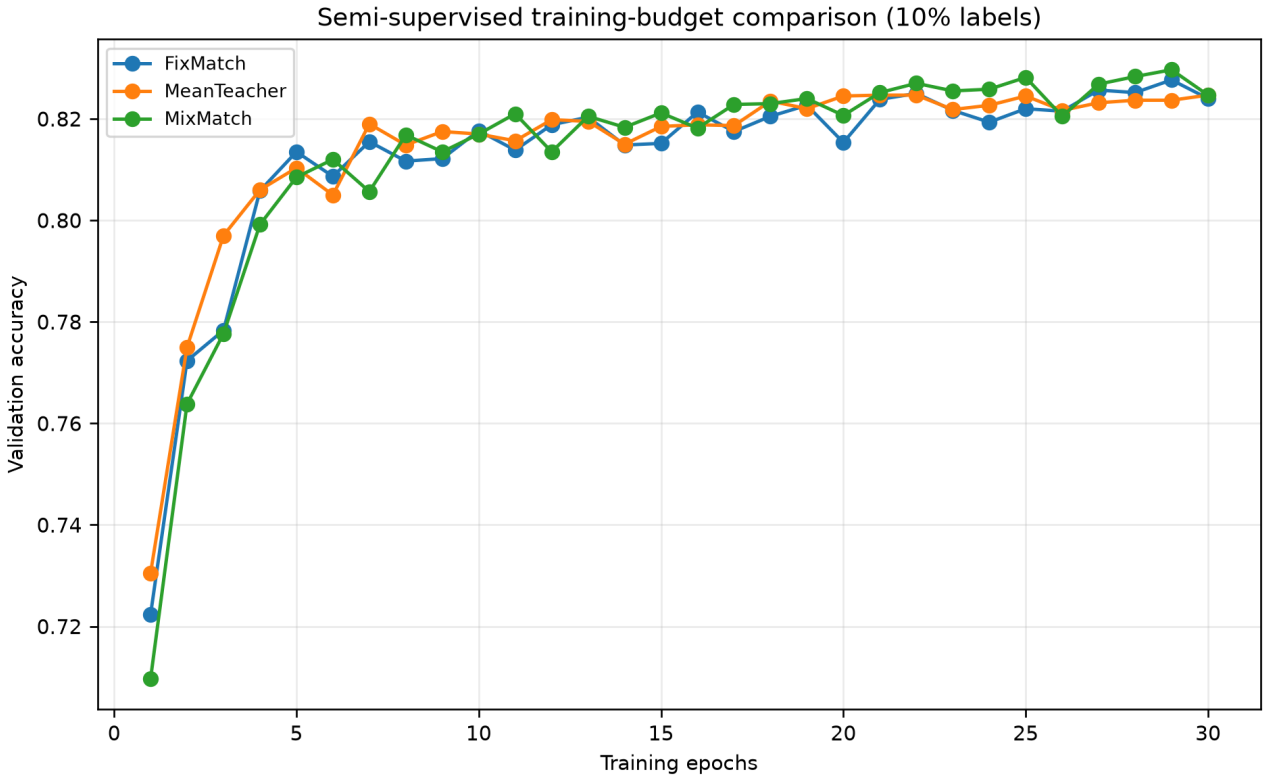


Data: results/raw/ssl_algorithms_test.csv

Interpretation: Algorithm and label-fraction effects are visible at the shared terminal budget.

Limitations: Three seeds per condition; supervised baseline only where matched and present in raw result tables.

Figure 12. Semi-supervised performance versus training budget

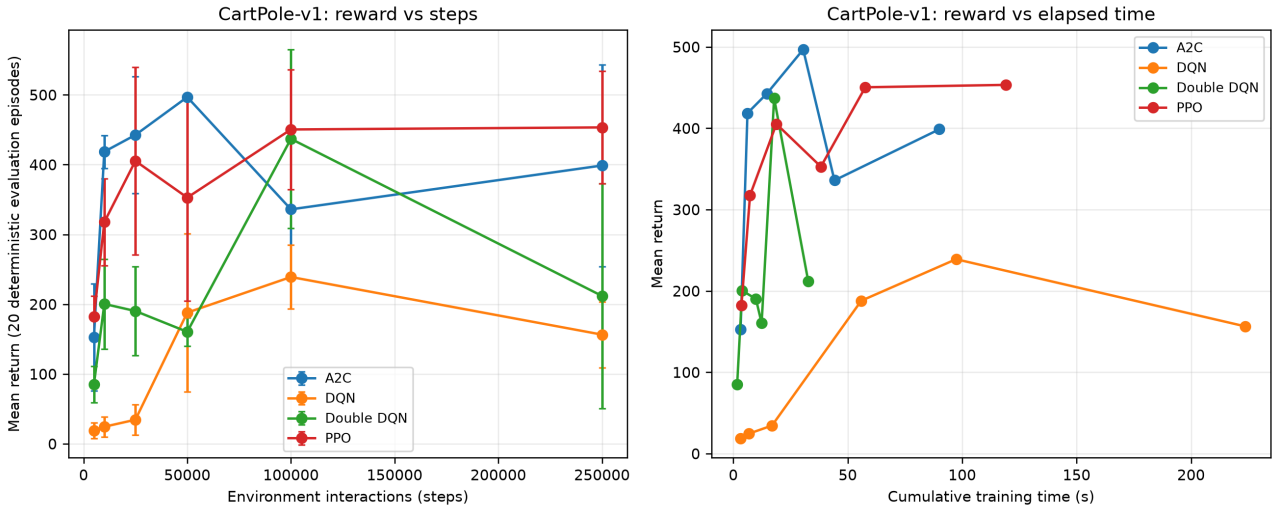


Data: results/raw/ssl_algorithms.csv

Interpretation: Methods have distinct trajectories at a common label fraction.

Limitations: Three seeds; task adaptations, not canonical implementations on standard SSL benchmark splits.

Figure 13. CartPole-v1 reward versus steps and time

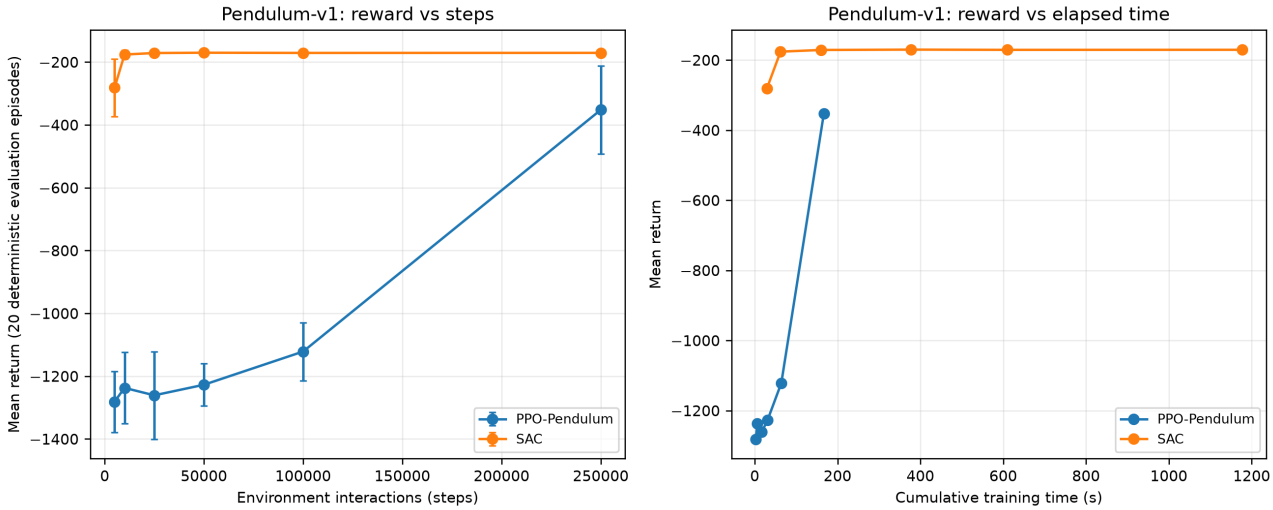


Data: results/raw/rl_scaling_results.csv

Interpretation: Algorithm-specific return trajectories are shown for CartPole-v1.

Limitations: Seed counts differ by algorithm; separate environments and reward scales; logged elapsed time is hardware/protocol specific.

Figure 14. Pendulum-v1 reward versus steps and time

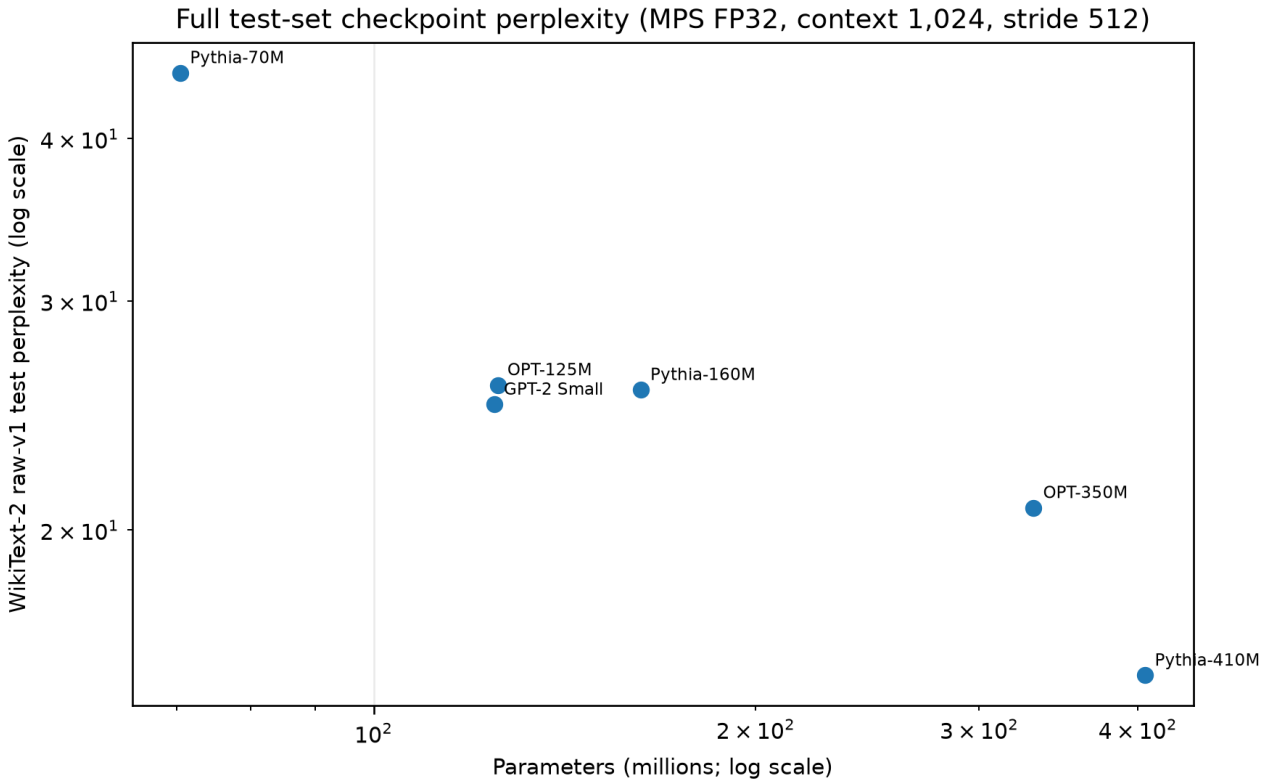


Data: results/raw/rl_scaling_results.csv

Interpretation: Algorithm-specific return trajectories are shown for Pendulum-v1.

Limitations: Seed counts differ by algorithm; separate environments and reward scales; logged elapsed time is hardware/protocol specific.

Figure 15. Full WikiText-2 perplexity versus parameter count

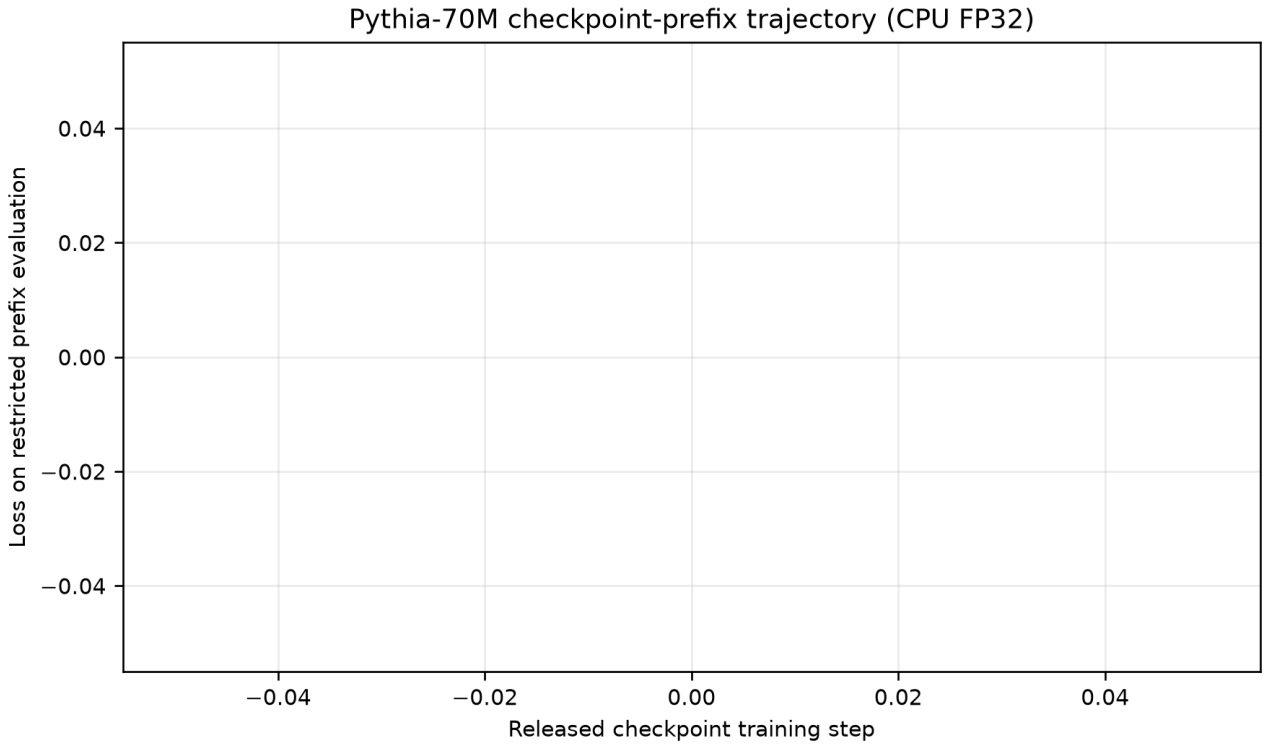


Data: results/llm/macos_full_wikitext2_evaluations.csv

Interpretation: Larger checkpoints in the Pythia series have lower measured perplexity under the matched local evaluation protocol.

Limitations: Six checkpoints across different model families; not matched pretraining compute, and tokenizer differences remain.

Figure 16. Pythia-70M checkpoint-prefix evaluation trajectory

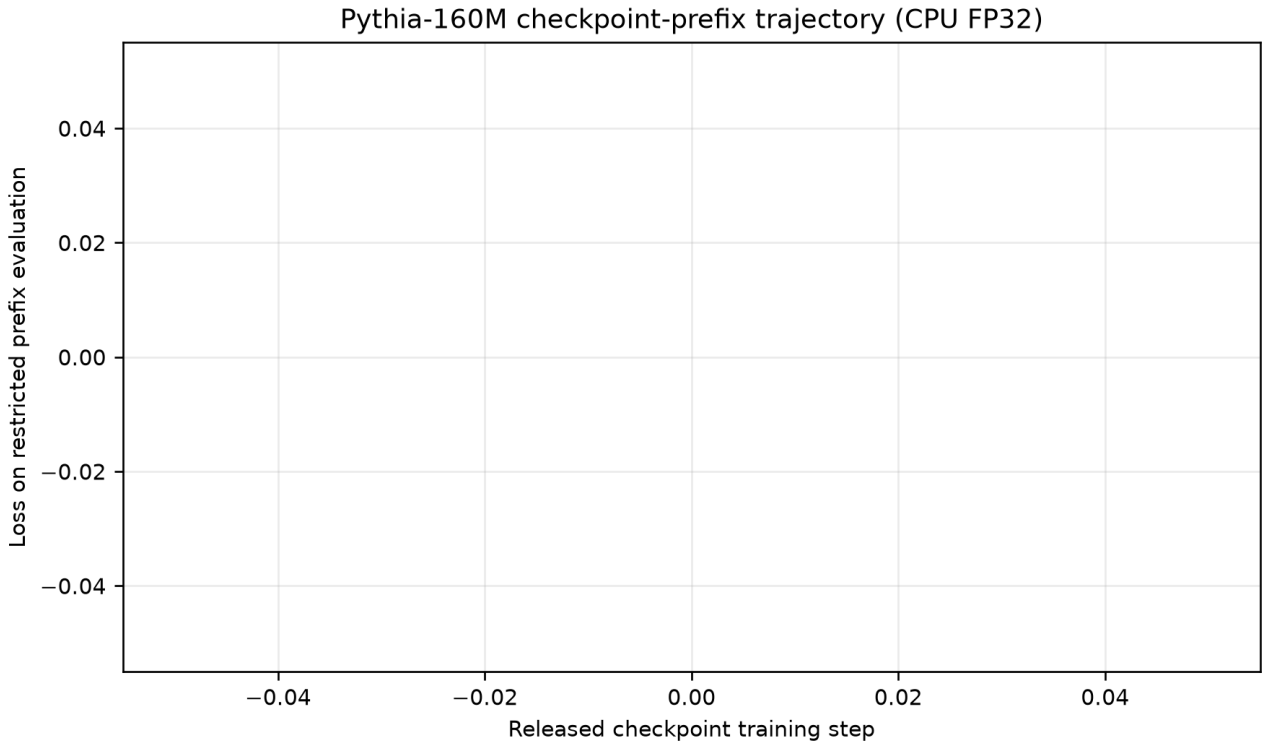


Data: results/llm/expanded_llm_evaluations.csv

Interpretation: Loss is evaluated at released checkpoints, including step 0, on a fixed restricted local text prefix.

Limitations: Single forward evaluations of selected released checkpoints on a small raw-text prefix; step-to-token mapping is published metadata, not measured local training compute.

Figure 17. Pythia-160M checkpoint-prefix evaluation trajectory

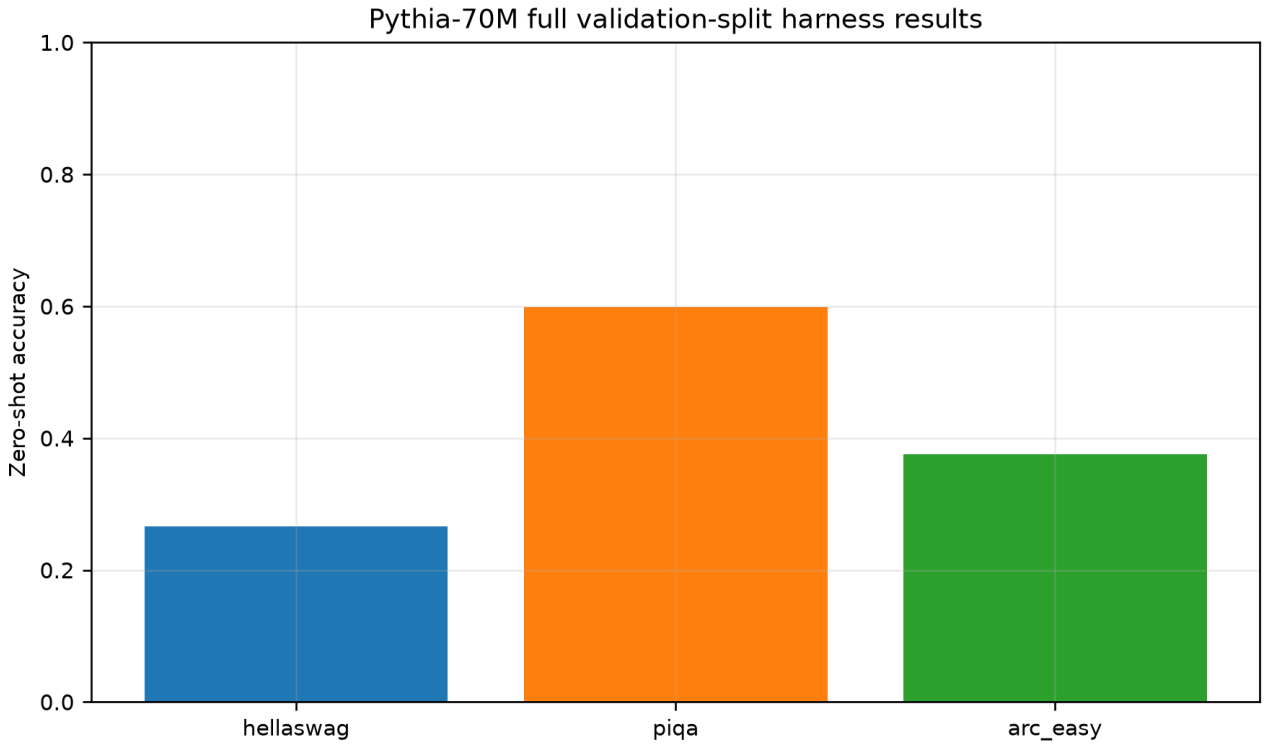


Data: results/llm/expanded_llm_evaluations.csv

Interpretation: Loss is evaluated at released checkpoints, including step 0, on a fixed restricted local text prefix.

Limitations: Single forward evaluations of selected released checkpoints on a small raw-text prefix; step-to-token mapping is published metadata, not measured local training compute.

Figure 18. Zero-shot benchmark results (full validation splits)

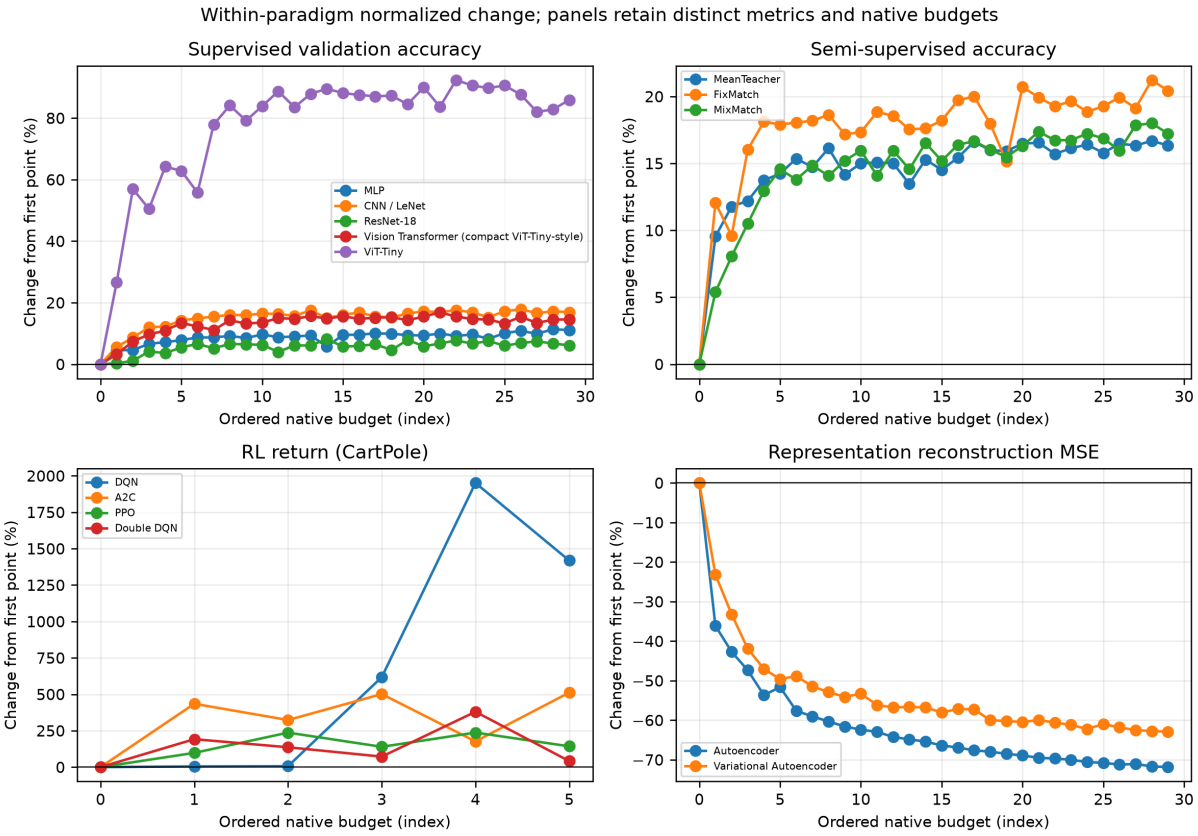


Data: results/llm/harness_full_summary.csv

Interpretation: Three full validation tasks were evaluated without few-shot examples.

Limitations: One model, three tasks; task scores are not pooled; smoke-test subsets are excluded.

Figure 19. Cross-paradigm within-task normalized changes

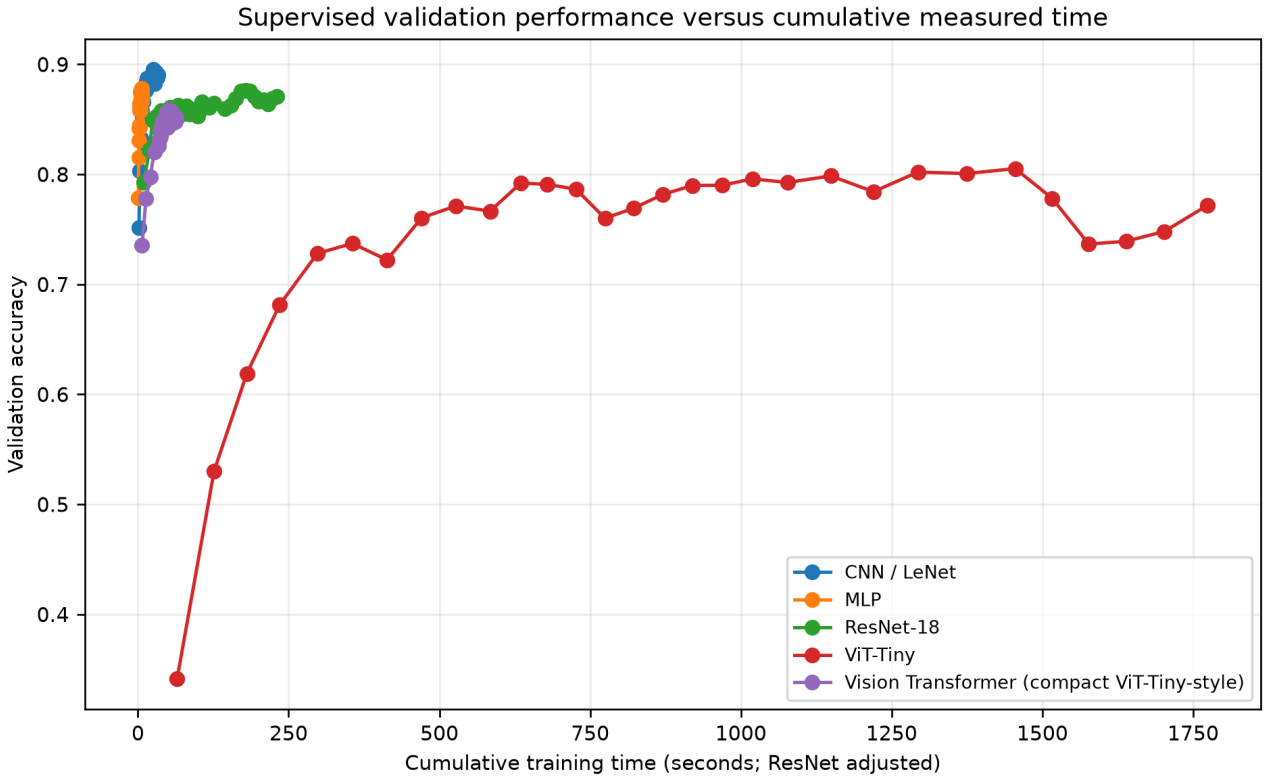


Data: multiple verified raw result tables

Interpretation: Each panel normalizes a metric only within its own series; no pooled score or shared compute axis.

Limitations: Index axis intentionally avoids pretending epochs, steps, and rewards are equivalent; percent change can be unstable near zero or for negative returns.

Figure 20. Supervised performance versus cumulative compute time

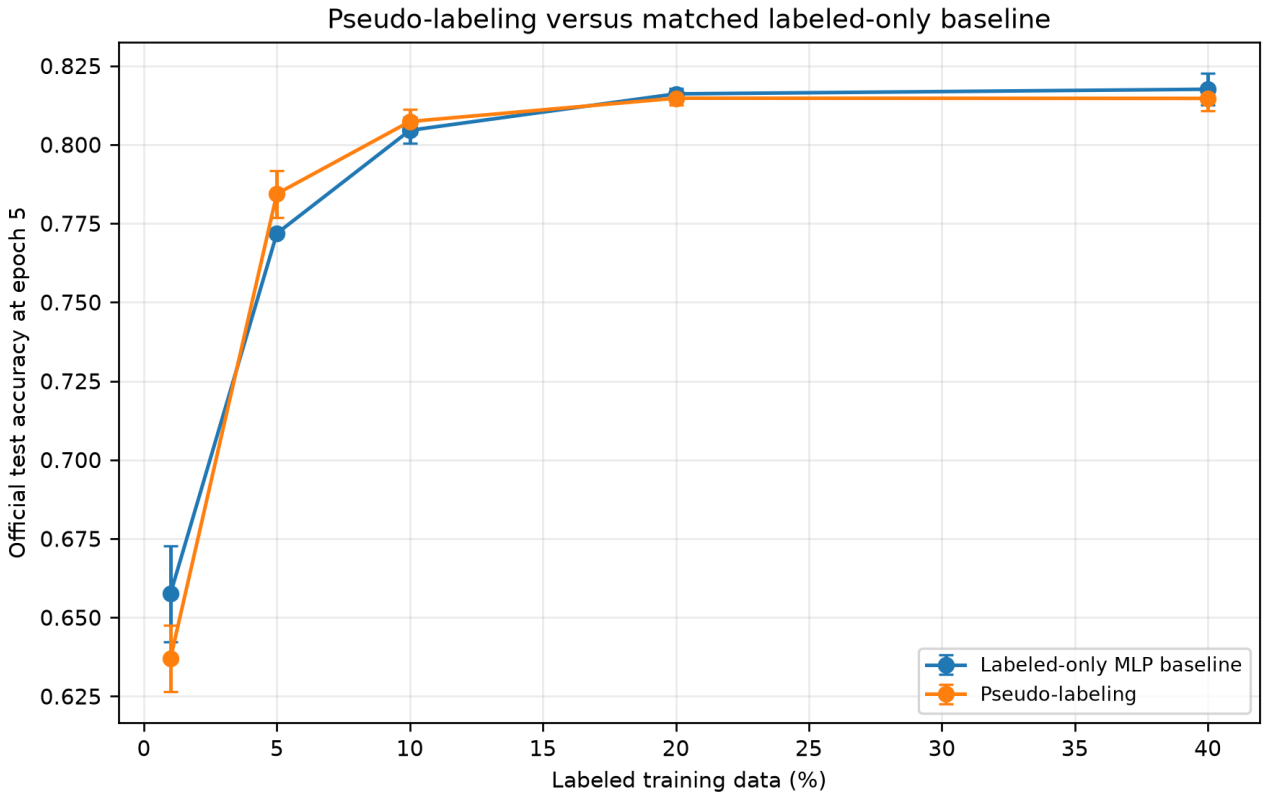


Data: results/raw/replicated_scaling_results.csv

Interpretation: Mean validation accuracy is plotted against mean cumulative wall time at each epoch.

Limitations: Time is elapsed runtime, not energy/FLOPs; ResNet seed-11 pause-like interval is removed only in this adjusted diagnostic.

Figure 21. Semi-supervised method versus supervised baseline

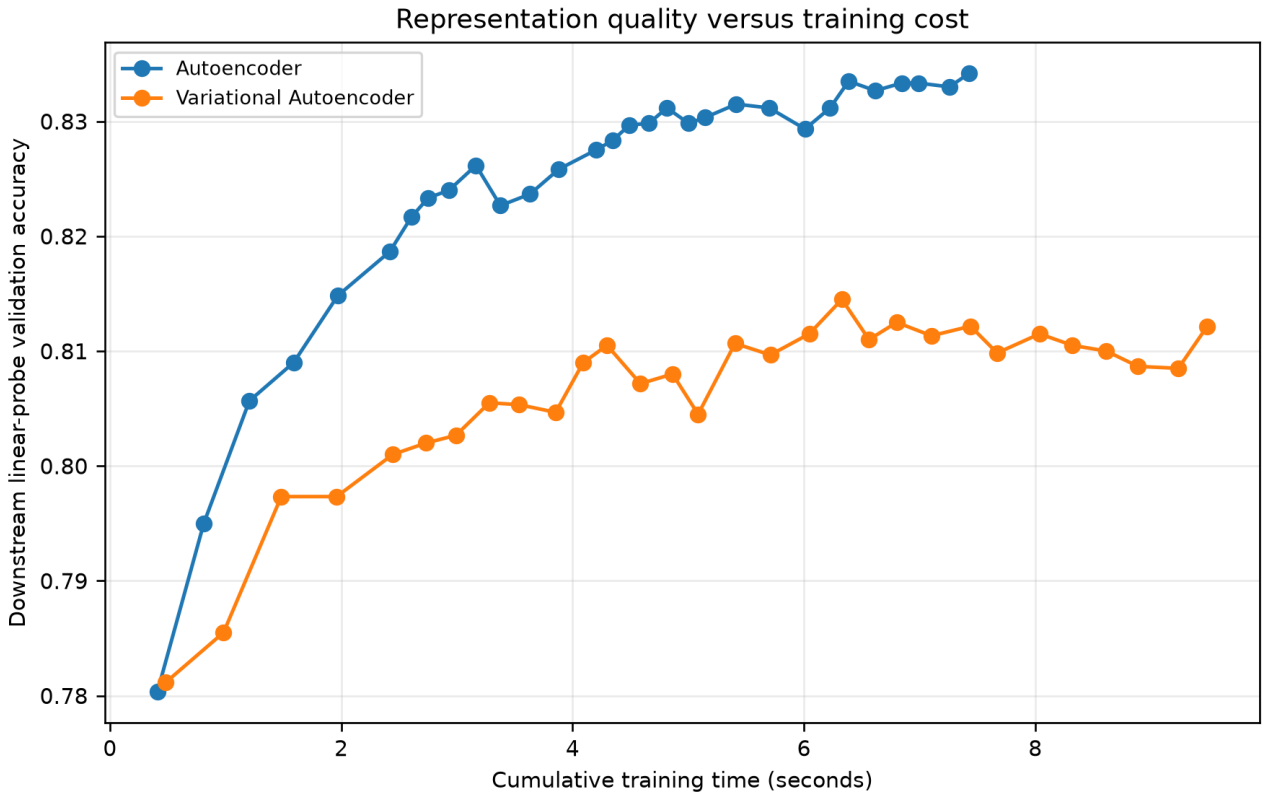


Data: results/raw/pseudolabel_test_results.csv

Interpretation: Pseudo-labeling is compared with the labeled-only MLP under matched fractions, seeds, and terminal epoch.

Limitations: Three seeds per fraction; this ablation is separate from the 30-epoch Mean Teacher/FixMatch/MixMatch runs.

Figure 22. Representation quality versus reconstruction-model training cost

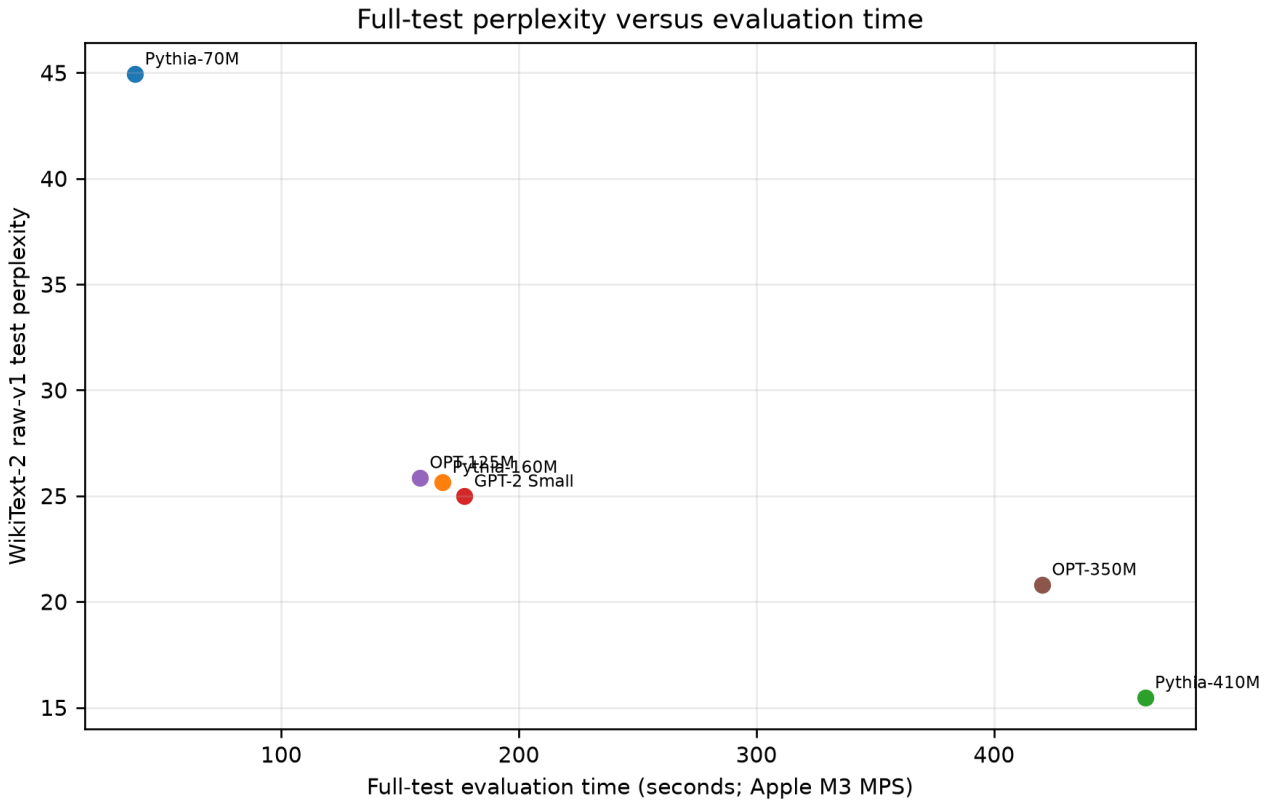


Data: results/raw/autoencoder_scaling.csv

Interpretation: Linear-probe accuracy is plotted against measured cumulative training time for each representation model.

Limitations: Three seeds; the probe is a separate downstream assessment and does not represent a universal representation metric.

Figure 23. Perplexity versus comparable full-test evaluation time

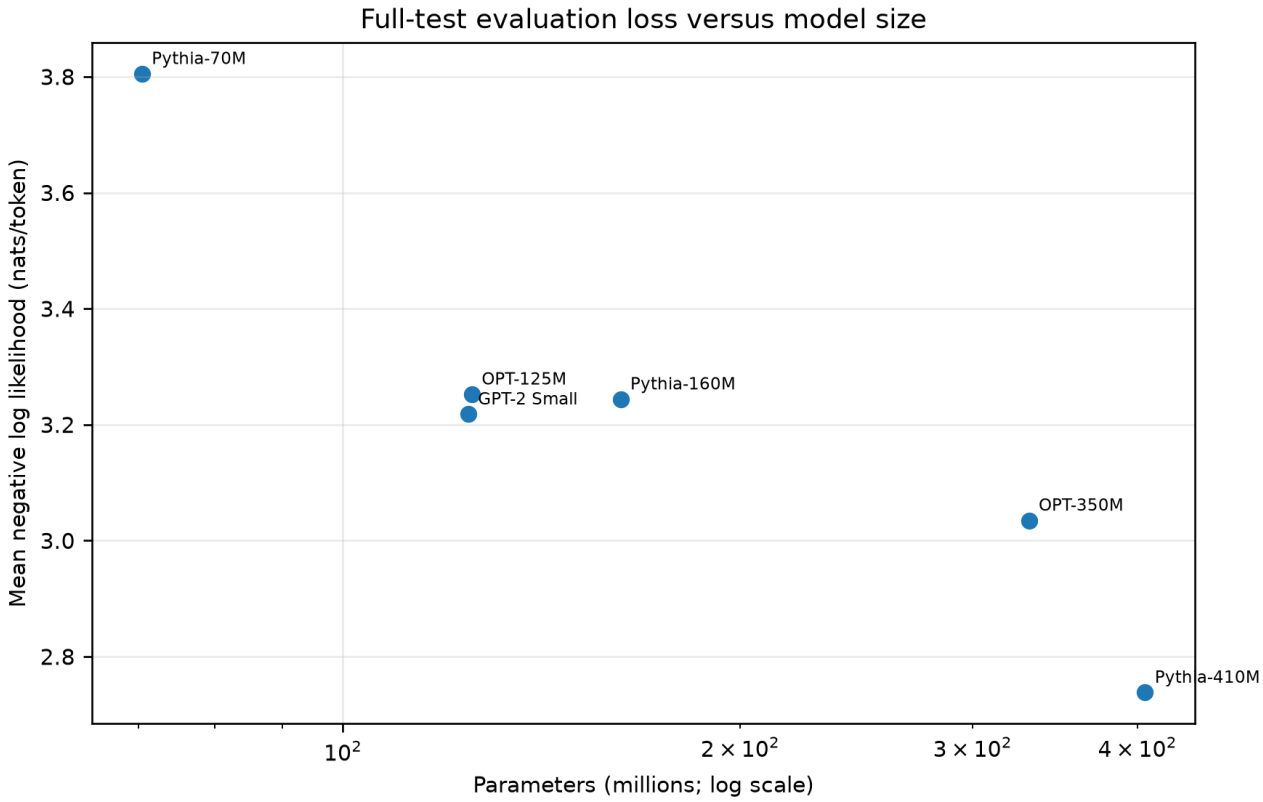


Data: results/llm/macos_full_wikitext2_evaluations.csv

Interpretation: The same local device, precision, context, stride, and corpus protocol permit descriptive inference-time comparison.

Limitations: Tokenizer-specific scored token counts differ; evaluation time is not original training compute and each model has one run.

Figure 24. Full-test evaluation loss versus parameter count

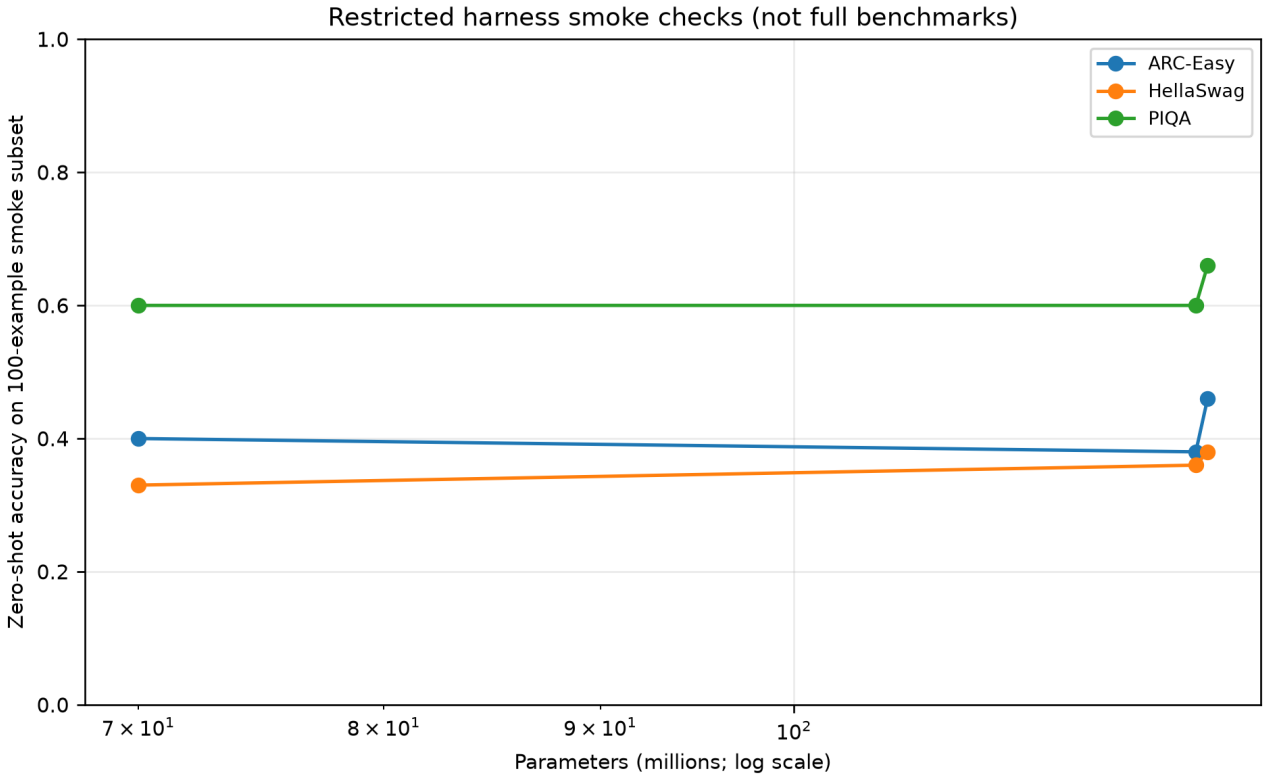


Data: results/llm/macos_full_wikitext2_evaluations.csv

Interpretation: Loss and parameter count are plotted for the same six local checkpoint evaluations.

Limitations: Cross-family training data and tokenization differ; one evaluation per model; size is not training compute.

Figure 25. Restricted zero-shot benchmark smoke tests



Data: results/llm/harness_summary.csv

Interpretation: Three checkpoints were evaluated on 100 examples per task; this view is explicitly separate from the full Pythia-70M validation evaluation.

Limitations: CPU FP32 smoke runs, limited examples; no claims about full-split comparative benchmark performance.